

 INSTITUTO SUPERIOR TECNOLÓGICO DE TURISMO Y PATRIMONIO YAVIRAC <i>¡Fortaleciendo Capacidades!</i>		
Nombre: Antony Guamán Anderson Narváez	Periodo Académico: 2026-1	Carrera: Desarrollo de Software
Asignatura: Devops	Fecha: 03/06/2026	Código:

Tema: Trabajo Colaborativo en parejas Pull Request y Merge

Control de Versiones

Definición:

El control de versiones es un sistema que registra todos los cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo, permitiendo recuperar versiones anteriores, comparar cambios y rastrear quién modificó qué y cuándo. Existen sistemas centralizados (como SVN) y distribuidos (como Git), siendo estos últimos los más usados en la actualidad.

Utilidad:

Permite trabajar en equipo sin sobrescribir el trabajo de otros, revertir errores rápidamente, mantener un historial completo del proyecto y ramificar el desarrollo para trabajar en funcionalidades de forma paralela e independiente.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Proyecto calculadora.py — sin control de versiones:
# calculadora_v1.py, calculadora_v2.py, calculadora_final.py ...
# Con Git, un solo archivo con historial completo:
$ git log --oneline calculadora.py
a3f1c2e feat: agregar multiplicacion y division
b8d4e01 feat: agregar estructura base de la calculadora
cla9f3d feat: commit inicial — README
```

Fuente: Chacon, S. & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

GitHub

Definición:

GitHub es una plataforma de alojamiento de repositorios Git en la nube, fundada en 2008 y adquirida por Microsoft en 2018. Además de alojar código, ofrece herramientas de colaboración como Pull Requests, Issues, wikis, revisión de código y automatización mediante GitHub Actions, convirtiéndose en la plataforma de desarrollo colaborativo más utilizada del mundo.

Utilidad:

Centraliza el código fuente accesible desde cualquier lugar, facilita la colaboración entre múltiples desarrolladores, permite automatizar pipelines de CI/CD, gestionar proyectos y documentar el software junto al código fuente.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Clonar el repositorio de GitHub a la máquina local:
$ git clone https://github.com/EstudianteA/actividad-git-colaborativa.git
$ cd actividad-git-colaborativa
# Verificar la conexión con el remoto:
$ git remote -v
origin https://github.com/EstudianteA/actividad-git-colaborativa.git (fetch)
origin https://github.com/EstudianteA/actividad-git-colaborativa.git (push)
```

Fuente: GitHub, Inc. (2024). GitHub Documentation. <https://docs.github.com>

Git

Definición:

Git es un sistema de control de versiones distribuido, de código abierto, creado por Linus Torvalds en 2005. A diferencia de los sistemas centralizados, cada desarrollador posee una copia completa del repositorio con todo su historial, permitiendo trabajar sin conexión a internet. Git gestiona los datos como una secuencia de instantáneas (snapshots) del proyecto.

Utilidad:

Gestiona el historial de proyectos de cualquier tamaño con velocidad y eficiencia. Permite crear ramas de forma instantánea, fusionar cambios, revertir errores y colaborar en equipos distribuidos globalmente sin necesidad de un servidor central.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Inicializar Git en el proyecto calculadora:
$ git init
$ git config --global user.name 'EstudianteA'
$ git config --global user.email 'estudianteA@uni.edu'
# Agregar calculadora.py y hacer el primer commit:
$ git add calculadora.py
$ git commit -m 'feat: agregar estructura base de la calculadora'
# Ver el estado del repositorio:
$ git status
$ git log --oneline
```

Fuente: Torvalds, L. (2005). Git — Fast Version Control System. <https://git-scm.com>

Repositorio Remoto

Definición:

Un repositorio remoto es una versión del proyecto alojada en un servidor accesible a través de internet o una red local (en este caso, GitHub). Actúa como punto central de sincronización para todos los miembros del equipo, siendo la fuente de verdad compartida del proyecto.

Utilidad:

Permite compartir el código entre múltiples colaboradores, actúa como respaldo automático del proyecto, sirve como punto de integración para los Pull Requests y activa los workflows de CI/CD ante cada cambio publicado.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Vincular el repo local con el remoto en GitHub:
$ git remote add origin https://github.com/EstudianteA/actividad-git-colaborativa.git
# Subir la rama main al repositorio remoto:
$ git push -u origin main
# Descargar cambios del remoto sin fusionar:
$ git fetch origin
# Descargar y fusionar cambios del remoto:
$ git pull origin main
```

Fuente: Chacon, S. & Straub, B. (2014). *Pro Git*. <https://git-scm.com/book/en/v2/Git-Basics-Working-with-Remotes>

Commit

Definición:

Un commit es una instantánea (snapshot) permanente del estado del proyecto en un momento determinado. Cada commit contiene: un hash SHA-1 único que lo identifica, el autor y fecha, un mensaje descriptivo, y una referencia al commit anterior, formando así la cadena de historial del proyecto.

Utilidad:

Registra de forma permanente los cambios con contexto descriptivo, permite revertir a cualquier estado anterior del proyecto, facilita identificar cuándo se introdujo un bug y quién realizó cada modificación en el código.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Estudiante A: commit de la estructura base
$ git add calculadora.py
$ git commit -m 'feat: agregar estructura base de la calculadora'
# Estudiante B: commit de nuevas funciones
$ git add calculadora.py
$ git commit -m 'feat: agregar multiplicacion y division'
# Ver historial con detalles:
$ git log --oneline --graph --all
* a3f1c2e (feature/multiplicacion-division) feat: agregar multiplicacion
* b8d4e01 (main) feat: agregar estructura base
```

Fuente: Conventional Commits. (2024). *A specification for commit messages*. <https://www.conventionalcommits.org>

Branch (Rama)

Definición:

Una rama es un puntero móvil a un commit específico que permite desarrollar funcionalidades de forma aislada del código principal. Crear una rama en Git es casi instantáneo porque solo crea un nuevo puntero de 41 bytes, sin duplicar archivos. La rama principal se llama 'main' (antes 'master').

Utilidad:

Permite trabajar en nuevas funcionalidades, correcciones o experimentos sin afectar el código estable de la rama principal, facilitando el desarrollo paralelo e independiente entre los miembros del equipo.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Estudiante A crea su rama:
$ git checkout -b feature/estructura-base
# Estudiante B crea su rama:
$ git checkout -b feature/multiplicacion-division
# Ver todas las ramas:
$ git branch -a
* feature/multiplicacion-division
  feature/estructura-base
  main
  remotes/origin/main
  remotes/origin/feature/estructura-base
```

Fuente: Atlassian. (2024). Git Branch. <https://www.atlassian.com/git/tutorials/using-branches>

Merge

Definición:

El merge (fusión) es la operación que integra los cambios de una rama en otra. Git analiza el historial común (commit base) de ambas ramas y aplica los cambios automáticamente cuando no hay conflictos, o bien marca los conflictos para resolución manual cuando dos ramas modificaron las mismas líneas.

Utilidad:

Permite integrar el trabajo desarrollado en ramas independientes al código principal, consolidando funcionalidades completas y revisadas en la rama de producción de forma controlada y documentada.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Después del merge de ambas ramas, calculadora.py queda:
def sumar(a, b):
    return a + b
def restar(a, b):
    return a - b
def multiplicar(a, b):      # <- aporte Estudiante B
    return a * b
def dividir(a, b):         # <- aporte Estudiante B
    if b == 0:
        return 'Error: division por cero'
    return a / b
```

Fuente: Chacon, S. & Straub, B. (2014). Pro Git — Branching and Merging. <https://git-scm.com/book/en/v2>

Pull Request

Definición:

Un Pull Request (PR) es una solicitud formal para integrar los cambios de una rama en otra dentro de plataformas como GitHub. No es una función de Git sino una herramienta de colaboración que permite revisar el código, agregar comentarios, solicitar modificaciones y aprobar cambios antes de fusionarlos.

Utilidad:

Garantiza la revisión del código antes de integrarlo a la rama principal, facilita la discusión técnica sobre implementaciones, documenta el razonamiento detrás de los cambios y activa automáticamente los pipelines de CI/CD para verificar la calidad.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Flujo completo de Pull Request:
# 1. Estudiante B publica su rama:
$ git push origin feature/multiplicacion-division
# 2. En GitHub: New Pull Request
#   Base: main <-- Compare: feature/multiplicacion-division
#   Título: 'feat: agregar multiplicacion y division a calculadora'
#   Reviewer: @EstudianteA
# 3. Estudiante A revisa los cambios en la pestaña 'Files changed'
# 4. Estudiante A aprueba y hace Merge
```

Fuente: GitHub Docs. (2024). About pull requests. <https://docs.github.com/en/pull-requests>

Integración Continua (CI)

Definición:

La Integración Continua (Continuous Integration) es una práctica donde los miembros del equipo integran su trabajo frecuentemente (al menos una vez al día). Cada integración es verificada por una compilación automatizada que incluye pruebas, para detectar errores lo antes posible en el ciclo de desarrollo.

Utilidad:

Reduce el riesgo de conflictos de integración complejos, detecta bugs tempranamente cuando son más fáciles y baratos de corregir, mejora la calidad del software y acelera el ciclo de retroalimentación para el equipo de desarrollo.

Ejemplo práctico — Proyecto Calculadora Python:

```
# .github/workflows/ci.yml — se ejecuta en cada push/PR:
name: CI - Pruebas Python
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: python calculadora.py
```

Fuente: Fowler, M. (2006). Continuous Integration. <https://martinfowler.com/articles/continuousIntegration.html>

Entrega Continua (Continuous Delivery)

Definición:

La Entrega Continua es una extensión de la Integración Continua que garantiza que el código siempre esté en un estado desplegable. Automatiza el proceso de lanzamiento hasta el entorno de preproducción (staging), pero requiere aprobación manual para el despliegue final a producción.

Utilidad:

Reduce el riesgo de despliegues, permite lanzar nuevas versiones de forma confiable en cualquier momento, acorta el tiempo de entrega de valor al cliente y aumenta la capacidad de respuesta ante los cambios del mercado.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Pipeline de Continuous Delivery para calculadora:
# Paso 1 (CI): Ejecutar pruebas automaticas
- run: python -m pytest tests/ -v
# Paso 2: Deploy automatico a staging
- name: Deploy a staging
  if: github.ref == 'refs/heads/main'
  run: echo 'Desplegando calculadora en servidor de staging...'
# Paso 3: Aprobacion MANUAL para ir a produccion
# (un miembro del equipo revisa staging y aprueba manualmente)
```

Fuente: Humble, J. & Farley, D. (2010). *Continuous Delivery*. Addison-Wesley

Despliegue Continuo (Continuous Deployment)

Definición:

El Despliegue Continuo es la práctica más avanzada del pipeline CI/CD. Cada cambio que supera todas las etapas automatizadas del pipeline se despliega directamente a producción sin ninguna intervención humana, eliminando el paso de aprobación manual presente en la Entrega Continua.

Utilidad:

Maximiza la velocidad de entrega de valor, elimina cuellos de botella en el proceso de lanzamiento, permite iterar rápidamente basándose en feedback real de usuarios y reduce la fricción entre los equipos de desarrollo y operaciones.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Continuous Deployment completo para calculadora:
name: CD - Despliegue Automatico
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: python calculadora.py # pruebas
      - run: echo 'Desplegado en produccion automaticamente!'
```

Fuente: Kim, G., Humble, J., Debois, P. & Willis, J. (2016). *The DevOps Handbook*. IT Revolution Press.

GitHub Actions

Definición:

GitHub Actions es la plataforma de automatización integrada en GitHub que permite definir flujos de trabajo (workflows) mediante archivos YAML ubicados en el directorio '.github/workflows/'. Estos workflows se ejecutan automáticamente ante eventos del repositorio como push, pull_request, o en horarios programados con cron.

Utilidad:

Automatiza las tareas de CI/CD directamente en GitHub sin servicios externos, ofrece acceso a miles de acciones reutilizables en el Marketplace, se integra nativamente con los Pull Requests y soporta múltiples sistemas operativos y lenguajes de programación.

Ejemplo práctico — Proyecto Calculadora Python:

```
# Archivo: .github/workflows/ci.yml
name: CI - Calculadora Python
on:
  push:
    branches: [ main, 'feature/*', 'fix/*' ]
  pull_request:
    branches: [ main ]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - name: Ejecutar calculadora
        run: python calculadora.py
```

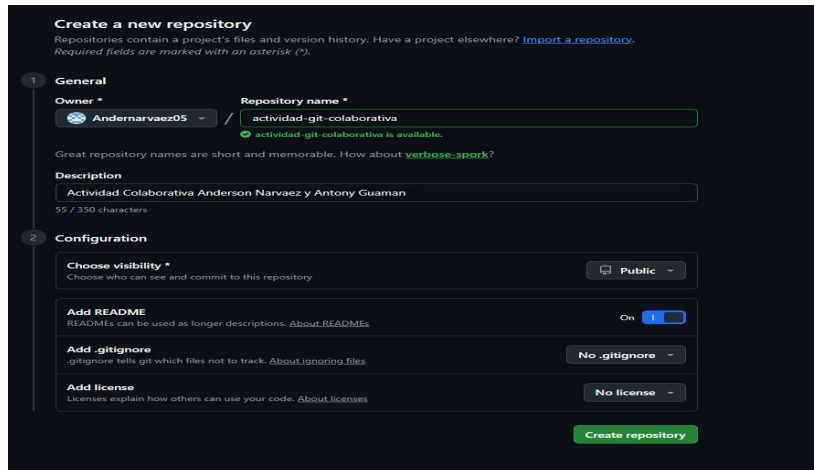
Fuente: GitHub Docs. (2024). GitHub Actions Documentation. <https://docs.github.com/en/act>

Práctica Colaborativa en Parejas

1) Creación del Repositorio en GitHub

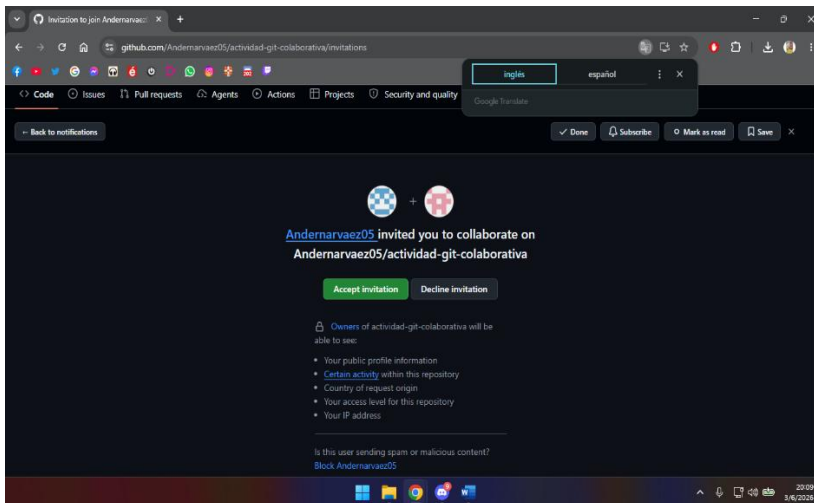
Anderson Narváez (EstudianteA) creó el repositorio público actividad-git-colaborativa en GitHub, con descripción "Actividad Colaborativa Anderson Narvaez y Antony Guaman", visibilidad pública y README activado. Luego envió una invitación a Antony Guamán (EstudianteB) para que colabore en el repositorio. Antony recibió la notificación en su cuenta y aceptó la invitación desde la pantalla de invitaciones de GitHub.

Formulario de creación del repositorio (Andernarvaez05):



The screenshot shows the 'Create a new repository' page on GitHub. It has two main sections: 'General' and 'Configuration'. In the 'General' section, the 'Owner' is 'Andernarvaez05' and the 'Repository name' is 'actividad-git-colaborativa'. The 'Description' is 'Actividad Colaborativa Anderson Narvaez y Antony Guaman'. In the 'Configuration' section, 'Choose visibility' is set to 'Public', 'Add README' is 'On', 'Add .gitignore' is 'No .gitignore', and 'Add license' is 'No license'. A green 'Create repository' button is at the bottom right.

Notificación de invitación recibida por TonyJ0711:



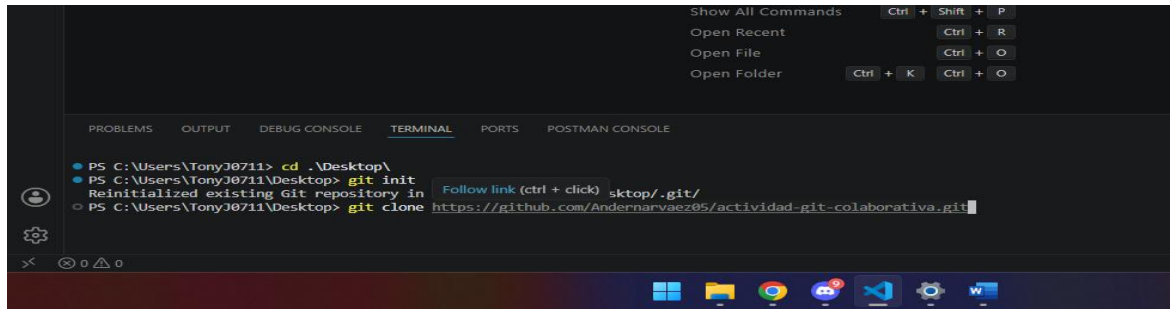
2) Clonación del Repositorio

Ambos integrantes clonaron el repositorio en sus máquinas locales usando VS Code con PowerShell. Anderson clonó desde su terminal y Antony desde la suya, ejecutando: `git clone https://github.com/Andernarvaez05/actividad-git-colaborativa.git`

Clonación realizada por Anderson:

```
PS C:\Users\WELCOME\documents> git clone https://github.com/Andernarvaez05/actividad-git-colaborativa.git
Cloning into 'actividad-git-colaborativa'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```


Clonación realizada por Antony:

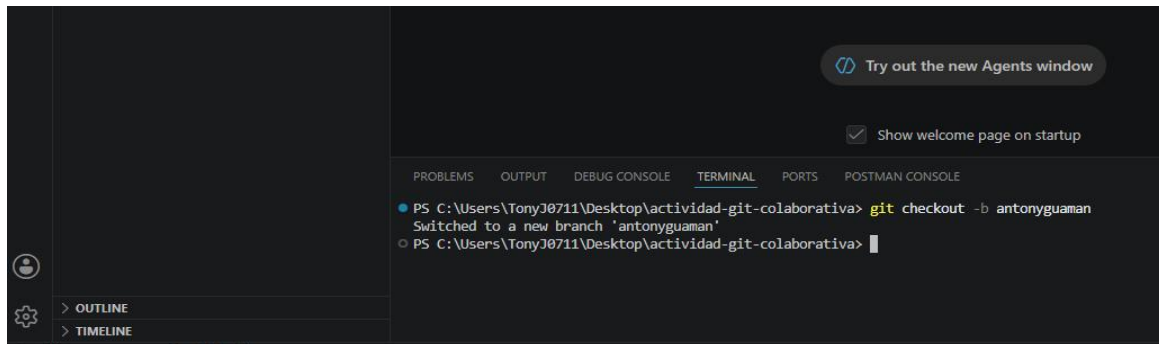


```
PS C:\Users\TonyJ0711> cd .\Desktop\  
PS C:\Users\TonyJ0711\Desktop> git init  
Reinitialized existing Git repository in sktop/.git/  
PS C:\Users\TonyJ0711\Desktop> git clone https://github.com/Andersonarvaez05/actividad-git-colaborativa.git
```

3) Creación de Ramas de Trabajo

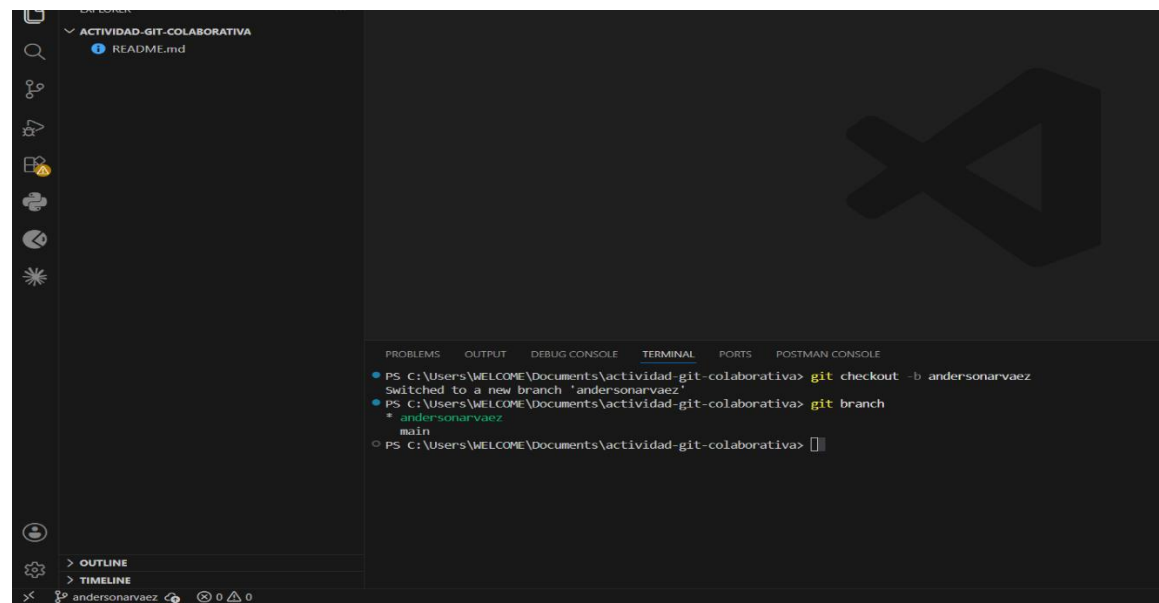
Cada integrante creó su propia rama de trabajo con el comando `git checkout -b`. Antony creó la rama `antonyguaman` y verificó con `git branch` que era la rama activa y Anderson creó la rama `andersonarvaez`.

Creación de rama `antonyguaman` :



```
PS C:\Users\TonyJ0711\Desktop\actividad-git-colaborativa> git checkout -b antonyguaman  
Switched to a new branch 'antonyguaman'  
PS C:\Users\TonyJ0711\Desktop\actividad-git-colaborativa>
```

Creación de rama `andersonarvaez` :

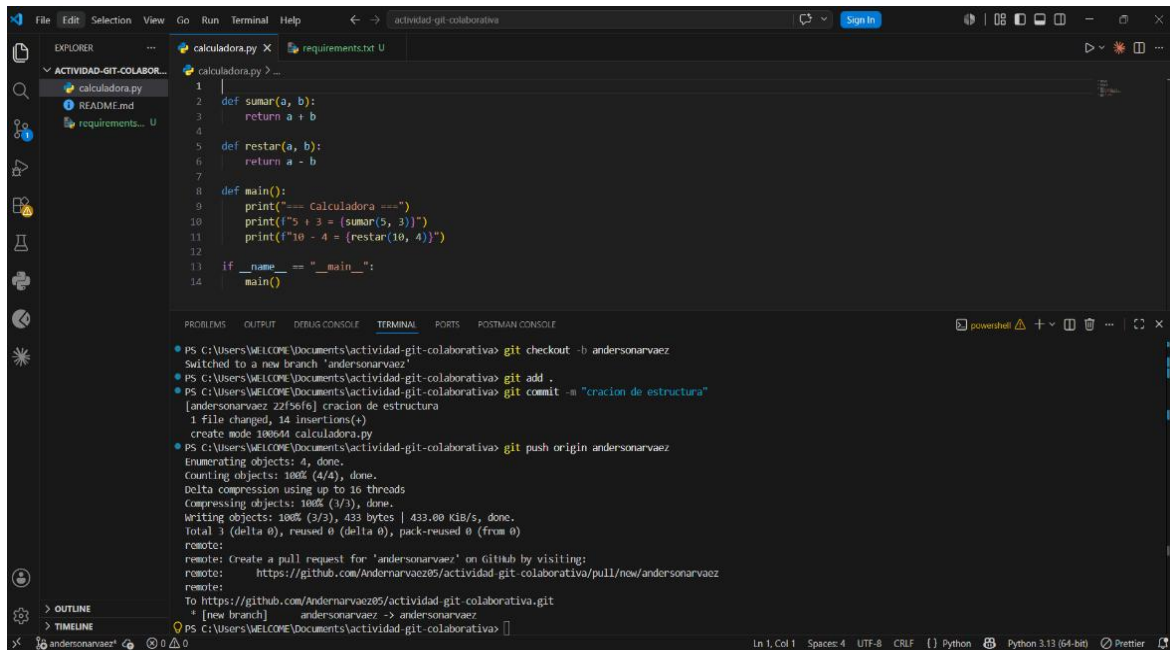


```
PS C:\Users\WELCOME\Documents\actividad-git-colaborativa> git checkout -b andersonarvaez  
Switched to a new branch 'andersonarvaez'  
PS C:\Users\WELCOME\Documents\actividad-git-colaborativa> git branch  
* andersonarvaez  
main  
PS C:\Users\WELCOME\Documents\actividad-git-colaborativa>
```

4) Commits Realizados por Ambos Integrantes

Anderson realizó el commit 'creación de estructura' agregando calculadora.py con las funciones sumar() y restar(). Antony realizó el commit 'agregar .github y archivo yml creando el workflow de GitHub Actions. Ambos usaron mensajes descriptivos que identifican claramente el propósito del cambio.

Commit de Anderson — calculadora.py (creación de estructura):



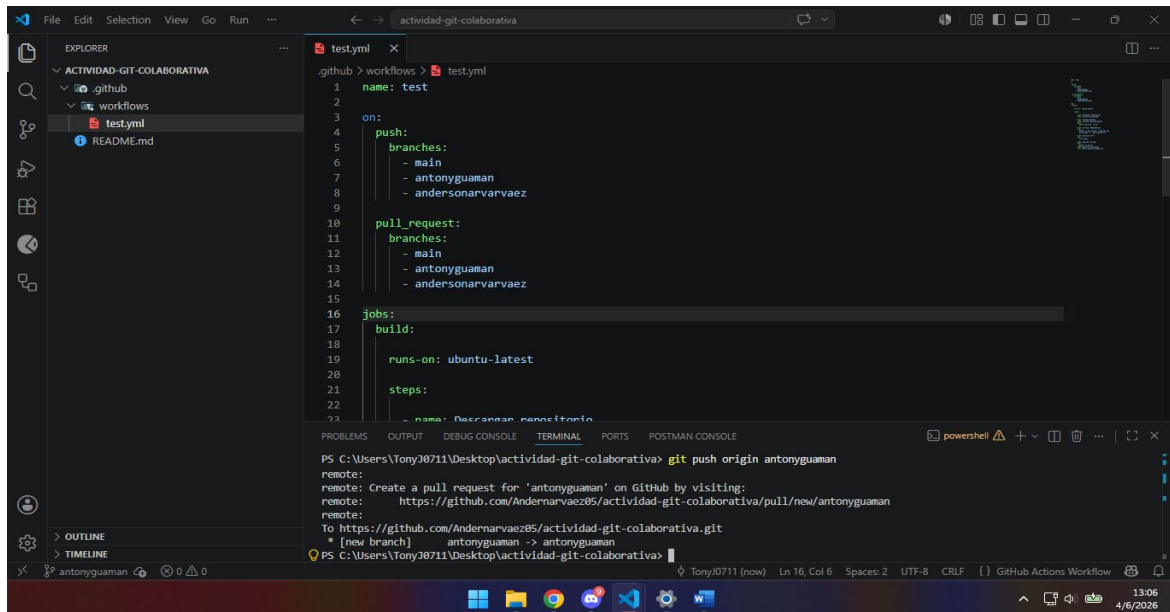
The screenshot shows the Visual Studio Code editor with a file named `calculadora.py` open. The file contains the following Python code:

```
1 |  
2 | def sumar(a, b):  
3 |     return a + b  
4 |  
5 | def restar(a, b):  
6 |     return a - b  
7 |  
8 | def main():  
9 |     print("=== Calculadora ===")  
10 |    print(f"5 + 3 = {sumar(5, 3)}")  
11 |    print(f"10 - 4 = {restar(10, 4)}")  
12 |  
13 | if __name__ == "__main__":  
14 |     main()
```

The terminal at the bottom shows the following commands and output:

```
PS C:\Users\WELCOME\Documents\actividad-git-colaborativa> git checkout -b andersonarvaez  
Switched to a new branch 'andersonarvaez'  
PS C:\Users\WELCOME\Documents\actividad-git-colaborativa> git add .  
PS C:\Users\WELCOME\Documents\actividad-git-colaborativa> git commit -m "creacion de estructura"  
[andersonarvaez 22f56f6] creacion de estructura  
1 file changed, 14 insertions(+)  
create mode 100644 calculadora.py  
PS C:\Users\WELCOME\Documents\actividad-git-colaborativa> git push origin andersonarvaez  
Enumerating objects: 4, done.  
Counting objects: 100% (4/4), done.  
Delta compression using up to 16 threads  
Compressing objects: 100% (3/3), done.  
Writing objects: 100% (3/3), 433 bytes | 433.00 KiB/s, done.  
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
remote:  
remote: Create a pull request for 'andersonarvaez' on GitHub by visiting:  
remote:   https://github.com/Andersonarvaez85/actividad-git-colaborativa/pull/new/andersonarvaez  
remote:  
To https://github.com/Andersonarvaez85/actividad-git-colaborativa.git  
 * [new branch]   andersonarvaez -> andersonarvaez  
PS C:\Users\WELCOME\Documents\actividad-git-colaborativa>
```

Commit de Antony — .github/workflows/test.yml (agregar .github y archivo yml):



The screenshot shows the Visual Studio Code editor with a file named `test.yml` open. The file contains the following YAML code:

```
1 | name: test  
2 |  
3 | on:  
4 |   push:  
5 |     branches:  
6 |       - main  
7 |       - antonyguaman  
8 |       - andersonarvaez  
9 |  
10 | pull_request:  
11 |   branches:  
12 |     - main  
13 |     - antonyguaman  
14 |     - andersonarvaez  
15 |  
16 | jobs:  
17 |   build:  
18 |     runs-on: ubuntu-latest  
19 |  
20 |     steps:  
21 |       - name: Descargan repositorio  
22 |  
23 |
```

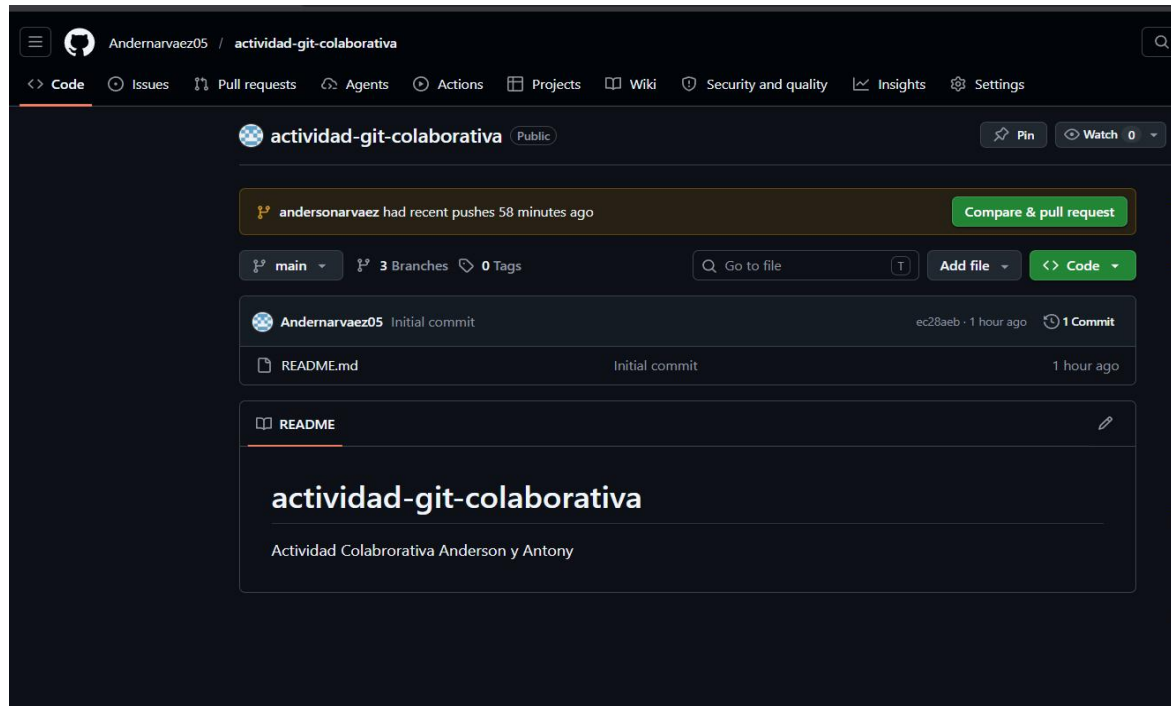
The terminal at the bottom shows the following commands and output:

```
PS C:\Users\Tony0711\Desktop\actividad-git-colaborativa> git push origin antonyguaman  
remote:  
remote: Create a pull request for 'antonyguaman' on GitHub by visiting:  
remote:   https://github.com/Andersonarvaez85/actividad-git-colaborativa/pull/new/antonyguaman  
remote:  
To https://github.com/Andersonarvaez85/actividad-git-colaborativa.git  
 * [new branch]   antonyguaman -> antonyguaman  
PS C:\Users\Tony0711\Desktop\actividad-git-colaborativa>
```

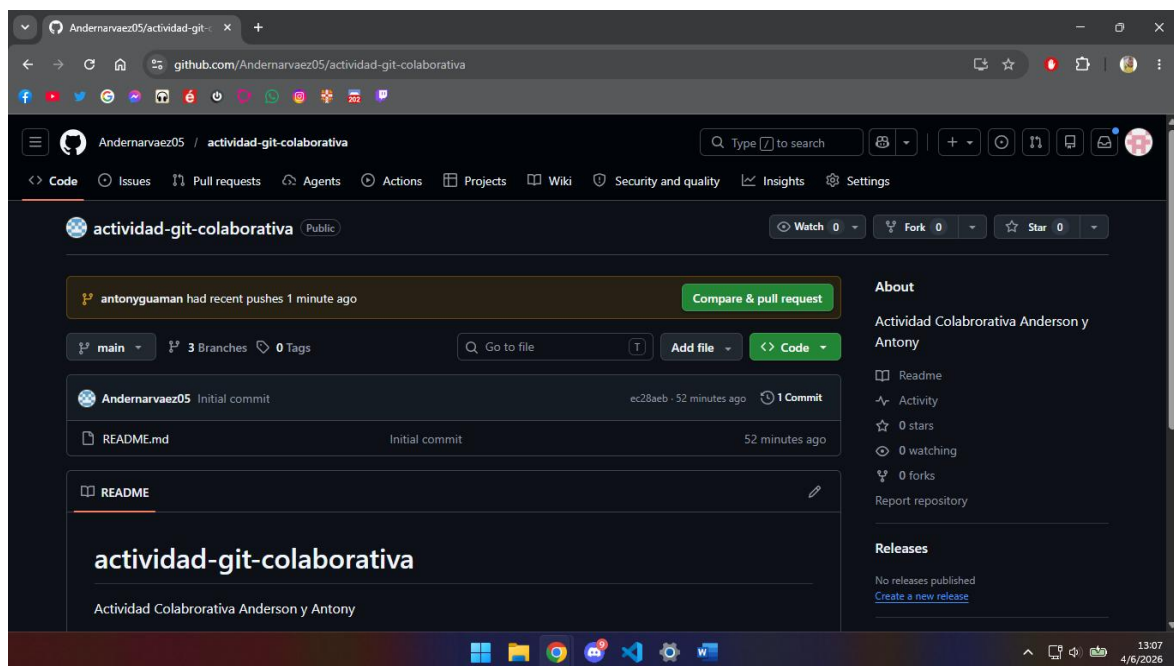
5) Publicación de Cambios al Repositorio Remoto

Cada integrante publicó su rama en el repositorio remoto con `git push origin [nombre-rama]`. Anderson ejecutó `git push origin andersonarvaez` y Antony `git push origin antonyguaman`. GitHub confirmó la recepción y sugirió automáticamente crear un Pull Request para cada rama publicada.

Push de Anderson — rama andersonarvaez publicada en GitHub:



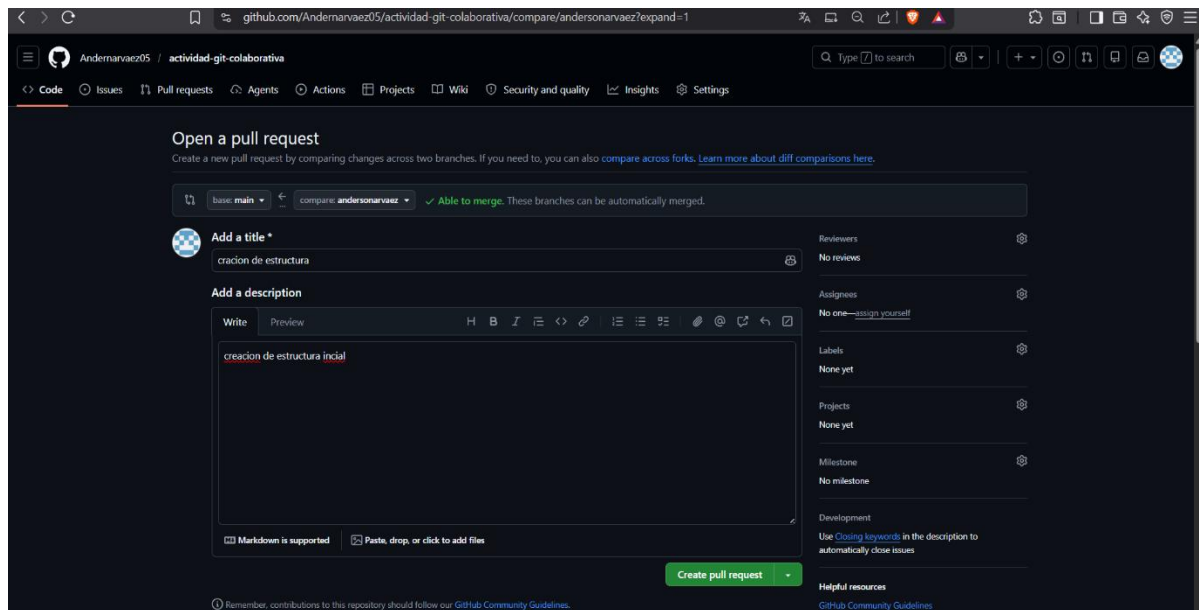
Push de Antony — rama antonyguaman publicada en GitHub:



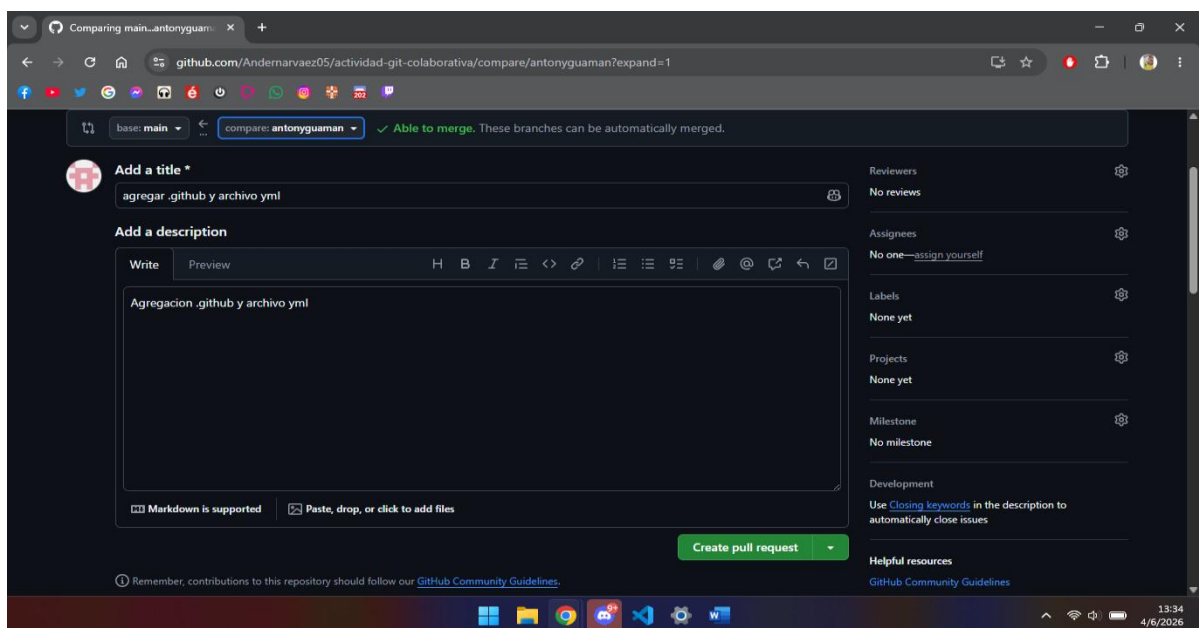
6) Creación de Pull Requests

Ambos integrantes crearon sus Pull Requests desde GitHub usando 'Compare & pull request'. Anderson abrió el PR 'creacion de estructura' (andersonarvaez → main), con diff de calculadora.py: 14 adiciones. Antony abrió el PR 'agregar .github y archivo yml' (antonyguaman → main), con diff de test.yml: 44 adiciones. Ambos PR indicaron 'Able to merge' al momento de creación.

Pull Request de Anderson (andersonarvaez → main):



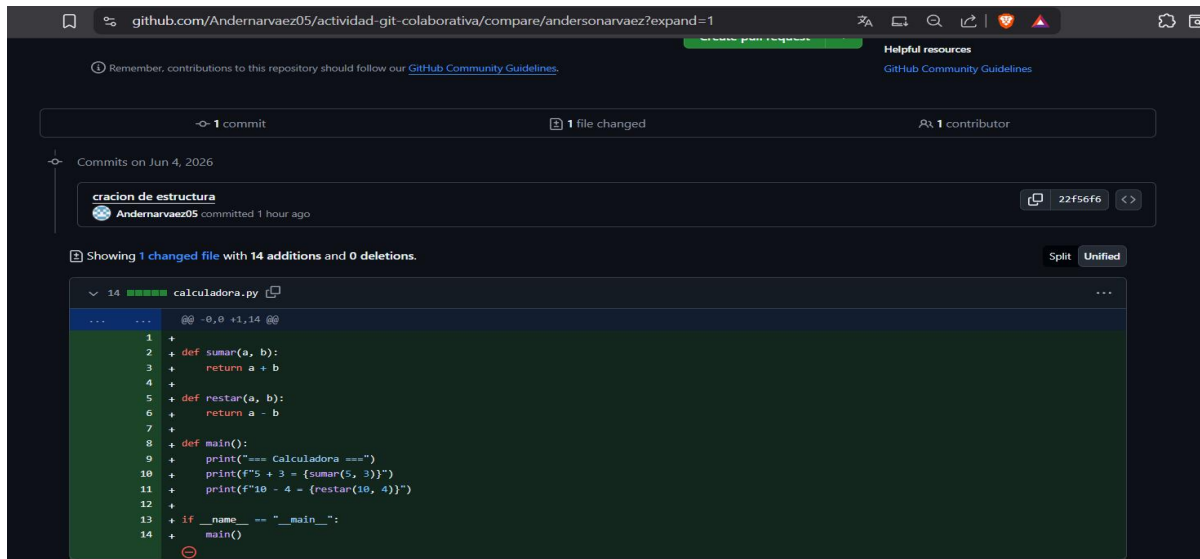
Pull Request de Antony (antonyguaman → main):



7) Revisión de Código

Cada integrante revisó el Pull Request del compañero desde la pestaña 'Files changed' en GitHub, donde se visualizan las líneas añadidas (en verde). El diff del PR de Anderson muestra calculadora.py con 14 líneas nuevas. El diff del PR de Antony muestra el test.yml completo con 44 adiciones.

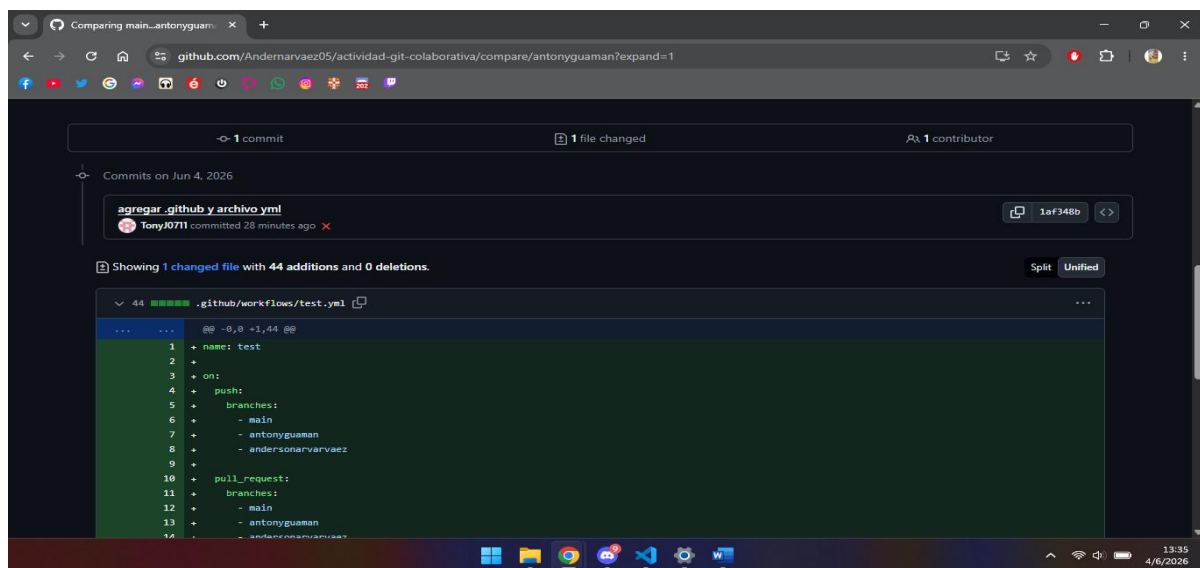
Revisión del diff — PR de Anderson (calculadora.py, 14 adiciones):



The screenshot shows a GitHub Pull Request interface for the repository 'Andernarvaez05/actividad-git-colaborativa'. The PR is titled 'cracion de estructura' and was committed by 'Andernarvaez05' 1 hour ago. It shows 1 file changed with 14 additions and 0 deletions. The diff for 'calculadora.py' is displayed, showing the following code:

```
@@ -0,0 +1,14 @@
1 +
2 + def sumar(a, b):
3 +     return a + b
4 +
5 + def restar(a, b):
6 +     return a - b
7 +
8 + def main():
9 +     print("=== Calculadora ===")
10 +     print(f"5 + 3 = {sumar(5, 3)}")
11 +     print(f"10 - 4 = {restar(10, 4)}")
12 +
13 + if __name__ == "__main__":
14 +     main()
```

Revisión del diff — PR de Antony (.github/workflows/test.yml, 44 adiciones):



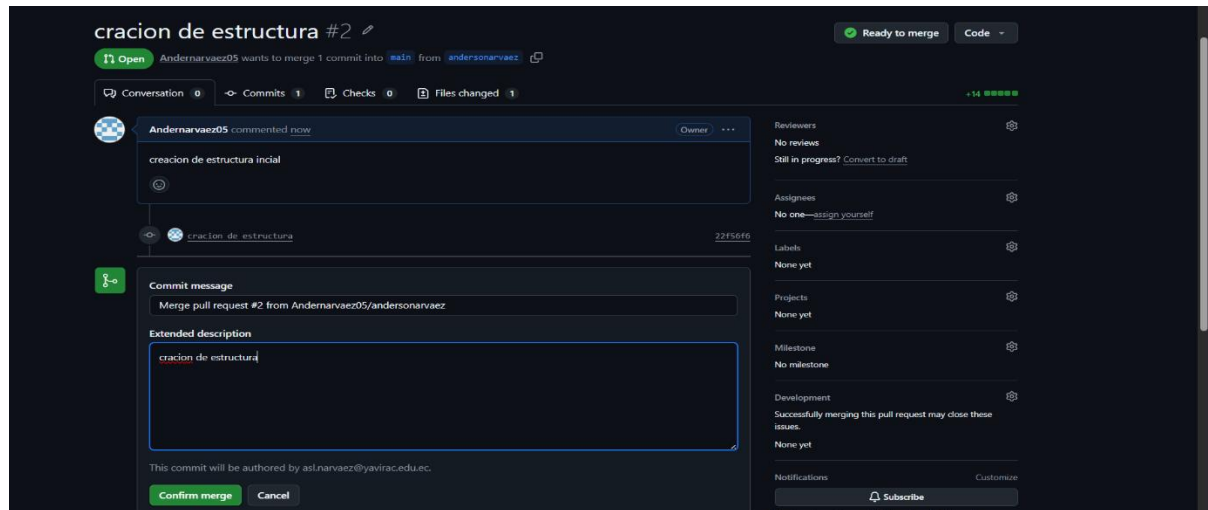
The screenshot shows a GitHub Pull Request interface for the repository 'Andernarvaez05/actividad-git-colaborativa'. The PR is titled 'agregar .github y archivo yml' and was committed by 'TonyJ0711' 28 minutes ago. It shows 1 file changed with 44 additions and 0 deletions. The diff for '.github/workflows/test.yml' is displayed, showing the following code:

```
@@ -0,0 +1,44 @@
1 + name: test
2 +
3 + on:
4 +   push:
5 +     branches:
6 +       - main
7 +       - antonyguaman
8 +       - andersonarvaez
9 +
10 +   pull_request:
11 +     branches:
12 +       - main
13 +       - antonyguaman
14 +       - andersonarvaez
```

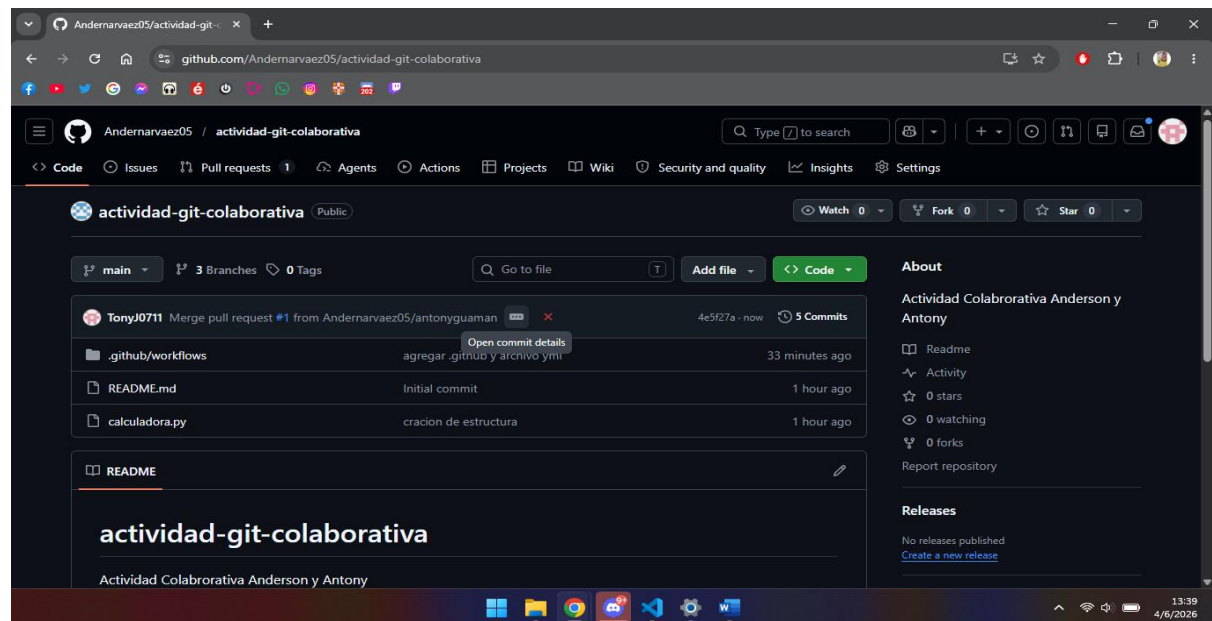
8) Aprobación de Pull Requests y Ejecución del Merge

Ambos Pull Requests fueron revisados, aprobados y fusionados a main. Se ejecutó 'Merge pull request' y 'Confirm merge' para cada uno. El historial registra: Merge pull request #2 (andersonarvaez, por Andernarvaez05) y Merge pull request #1 (antonyguaman, por TonyJ0711). Tras los merges, main quedó con 5 commits y los archivos calculadora.py, README.md y .github/workflows/test.yml integrados. Anderson ejecutó git pull origin main para sincronizar su copia local con los cambios de Antony.

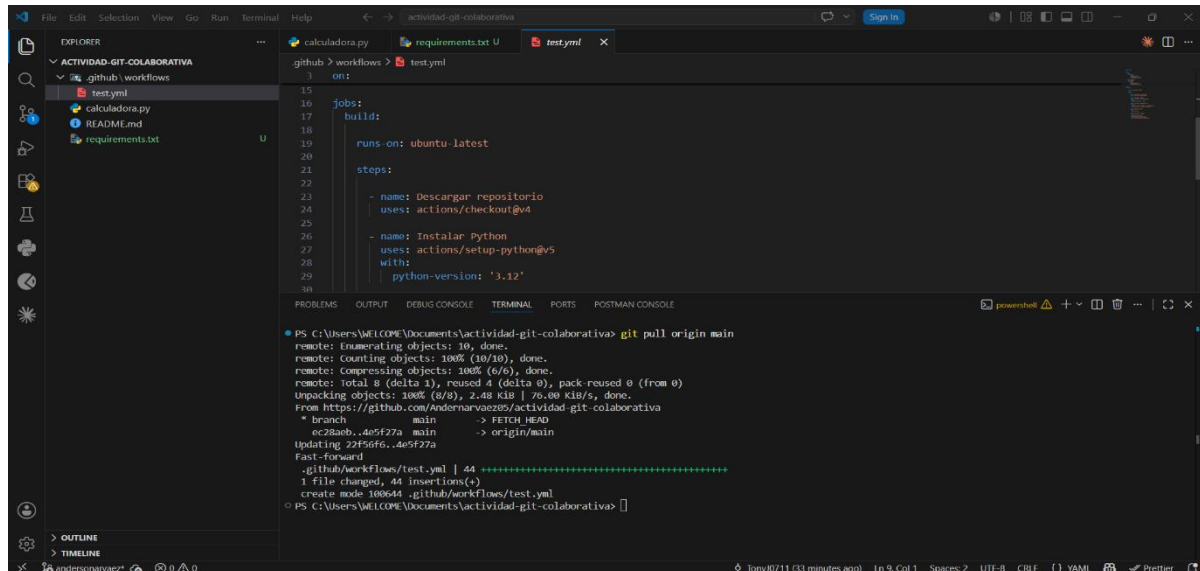
Merge del PR #1 ejecutado desde GitHub:



Repositorio main después de los merges (5 commits, 2 contributors):



git pull origin main ejecutado por Anderson para sincronizar localmente:

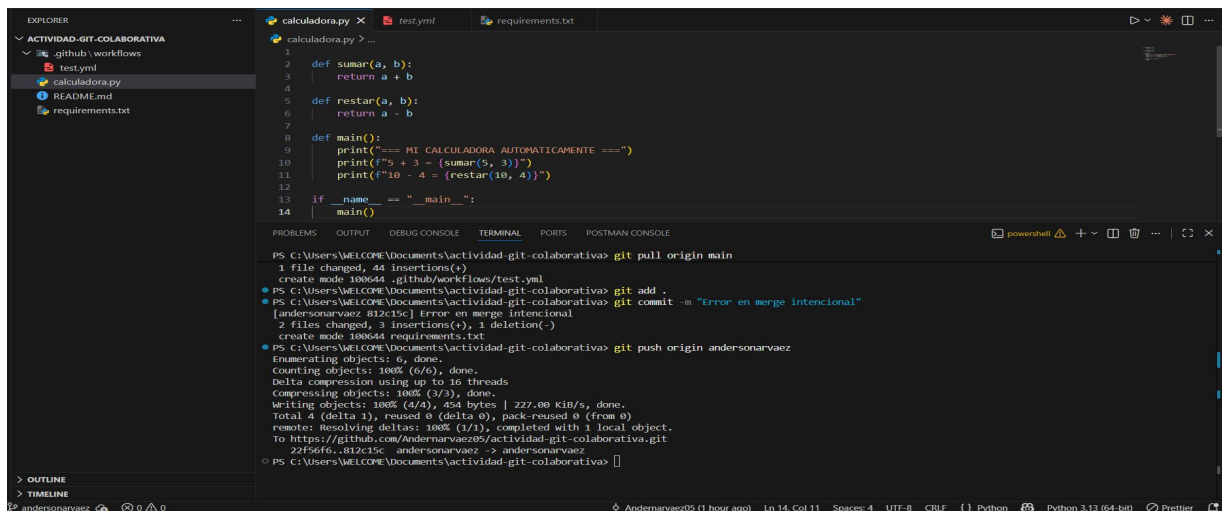


The screenshot shows the VS Code interface with a file explorer on the left showing 'ACTIVIDAD-GIT-COLABORATIVA' with files like 'calculadora.py', 'README.md', and 'requirements.txt'. The main editor shows a GitHub Actions workflow file 'test.yml' with a 'build' job. The terminal at the bottom shows the output of a 'git pull origin main' command, indicating a successful fast-forward merge of the main branch.

9) Generación y Resolución del Conflicto de Integración

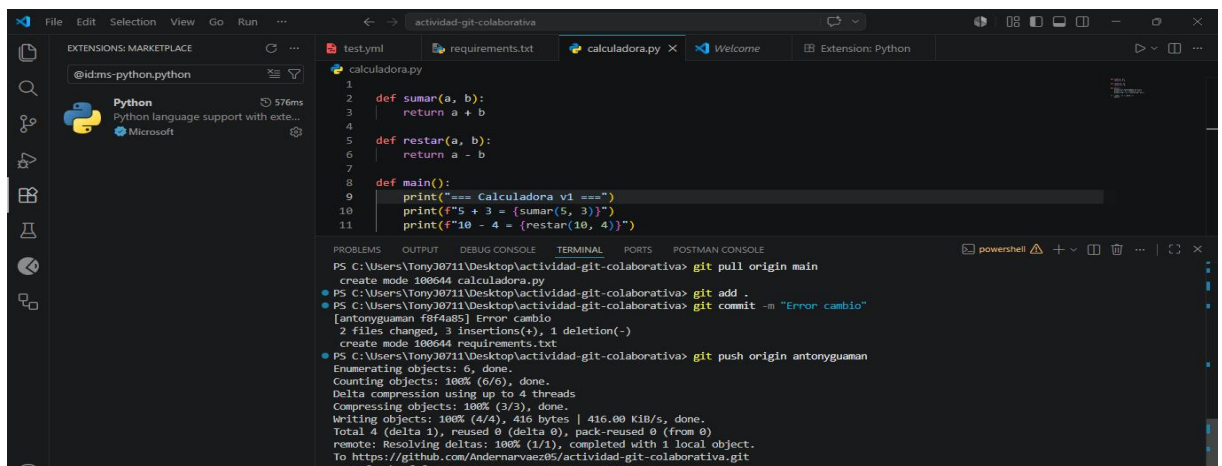
Para demostrar la resolución de conflictos, ambos editaron intencionalmente la misma línea del archivo calculadora.py (línea 9 — el mensaje de bienvenida). Anderson cambió la línea a `print("=== MI CALCULADORA AUTOMATICAMENTE ===")` y Antony a `print("=== Calculadora v1 ===")`. Ambos hicieron push de sus cambios a sus ramas. Al intentar el segundo merge en GitHub, el workflow de CI detectó el problema y mostró 'All checks have failed'. VS Code mostró el conflicto en vista comparativa (rojo vs verde). Anderson resolvió el conflicto localmente dejando `print("=== Calculadora ===")`, hizo commit 'Merge Final' y git push origin main.

Push de Anderson con el cambio intencional en la línea 9:



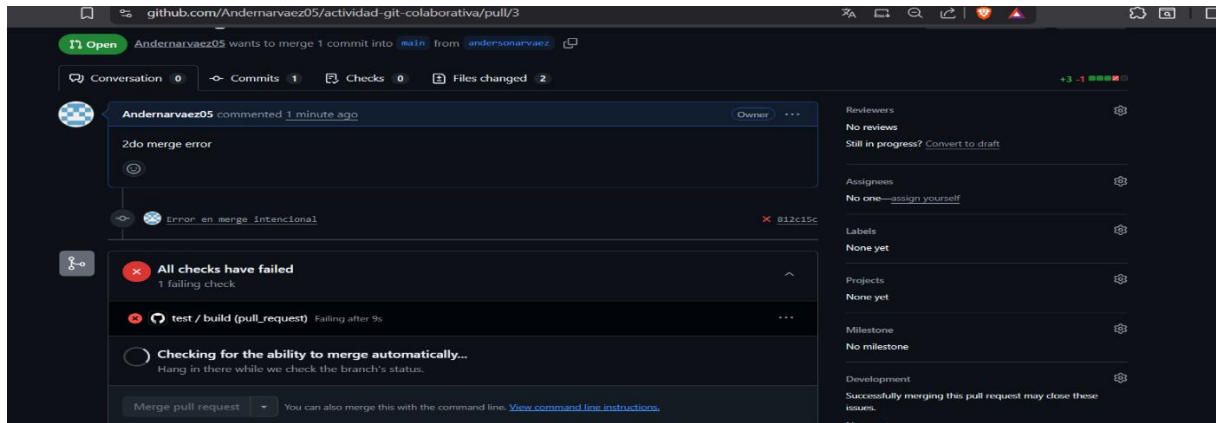
The screenshot shows the VS Code interface with a file explorer on the left. The main editor shows the 'calculadora.py' file with a conflict on line 9. The terminal at the bottom shows the output of a 'git pull origin main' command, which failed due to a merge conflict. It then shows the output of a 'git add .' command, which staged the changes. Finally, it shows the output of a 'git push origin andersonarvaez' command, which pushed the changes to the remote repository.

Push de Antony con su cambio intencional en la misma línea 9:

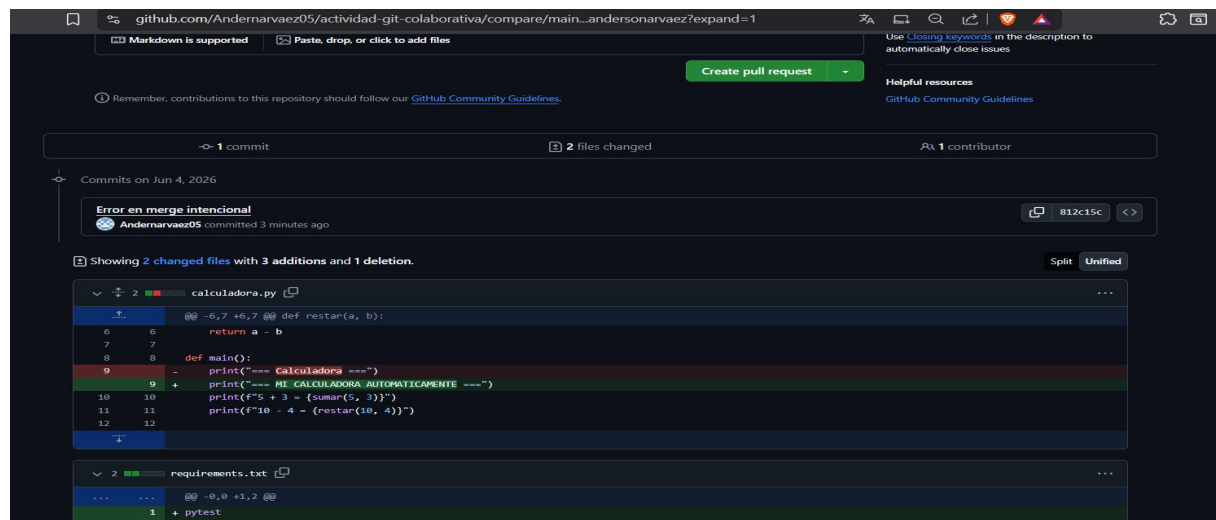


The screenshot shows the VS Code interface with a file explorer on the left. The main editor shows the 'calculadora.py' file with a conflict on line 9. The terminal at the bottom shows the output of a 'git pull origin main' command, which failed due to a merge conflict. It then shows the output of a 'git add .' command, which staged the changes. Finally, it shows the output of a 'git push origin antonyguaman' command, which pushed the changes to the remote repository.

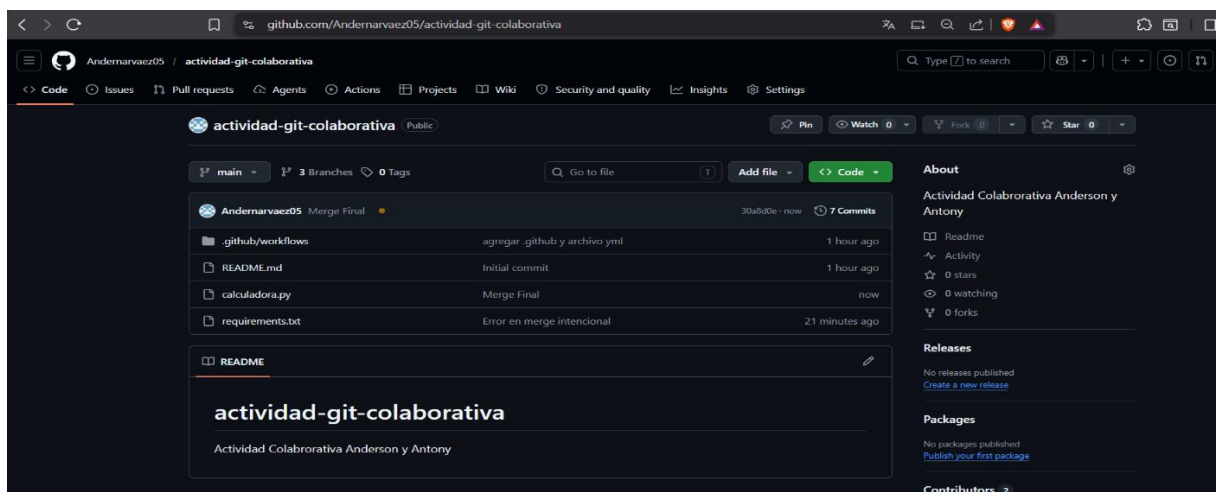
GitHub detecta el conflicto 'All checks have failed':



Conflicto en el PR:



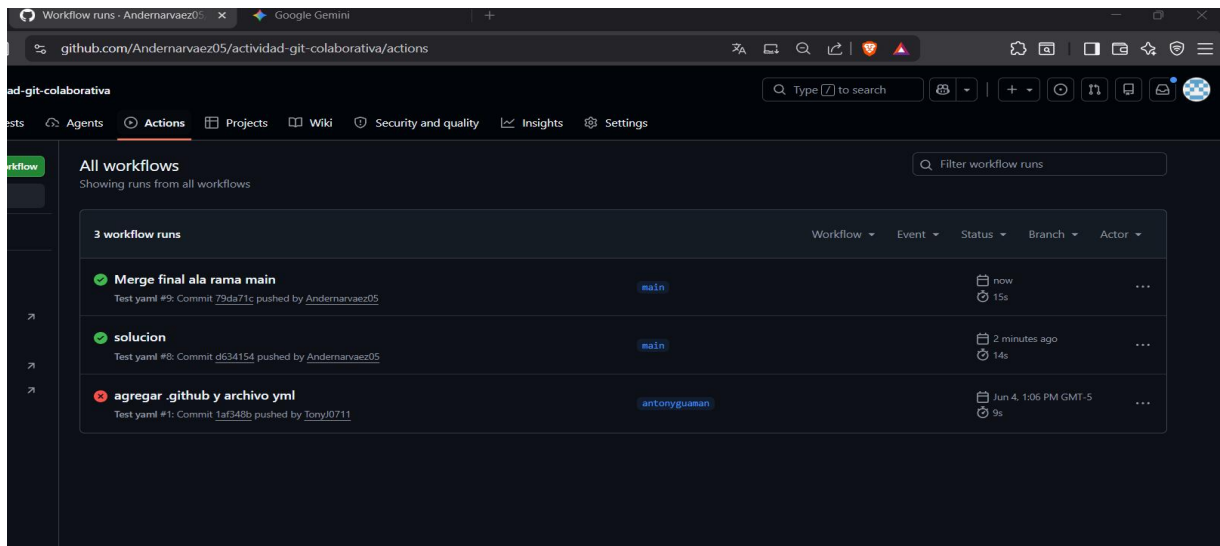
Commit 'Merge Final' y push a main:



10) Ejecución Exitosa del Workflow de GitHub Actions

El archivo `.github/workflows/test.yml` creado por Antony define el workflow 'test' que se ejecuta automáticamente ante cada push y pull request. 'Merge final ala rama main', ambas en rama main se completaron exitosamente, confirmando que el pipeline de integración continua funciona correctamente.

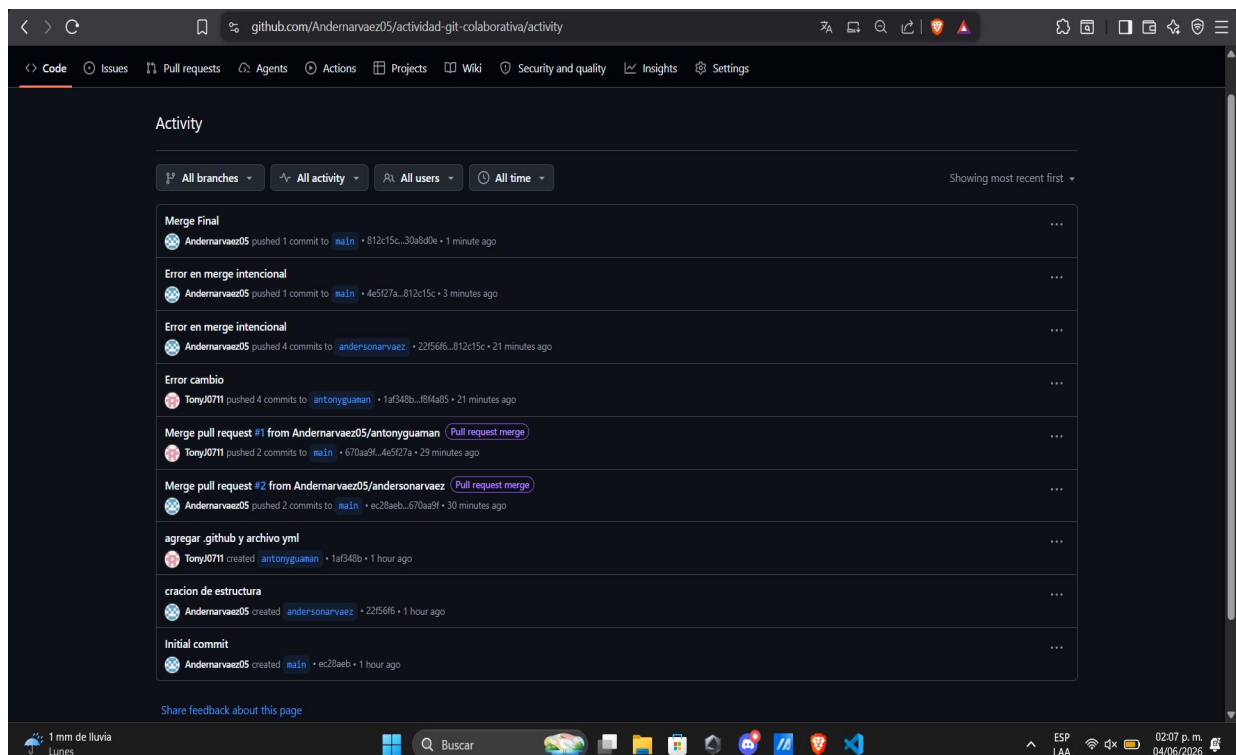
GitHub Actions:



11) Estado Final del Repositorio

El historial de actividad del repositorio registra cronológicamente todo el trabajo colaborativo: Initial commit → cracion de estructura (Andernarvaez05, andersonarvaez) → agregar `.github` y archivo yml (TonyJ0711, antonyguaman) → Merge PR #2 andersonarvaez → Merge PR #1 antonyguaman → Error cambio (TonyJ0711) → Error en merge intencional (Andernarvaez05) → Merge Final (Andernarvaez05, main). La sección Contributors muestra 2 colaboradores activos: Andernarvaez05 y TonyJ0711.

Historial completo de actividad del repositorio:



Información del Grupo

Integrantes: Anderson Narváez y Antony Guamán

Carrera y Paralelo: Desarrollo de Software — 5-VE-B

URL del Repositorio: <https://github.com/Andernarvaez05/actividad-git-colaborativa>

Conclusiones

Anderson Narváez:

Esta actividad me permitió comprender de forma práctica cómo funciona el trabajo colaborativo con Git y GitHub. Aprendí a crear y gestionar ramas de trabajo, lo que facilita el desarrollo paralelo sin interferir con el código del compañero. El proceso de Pull Request me resultó muy valioso porque obliga a revisar el código antes de integrarlo, mejorando la calidad del producto final. La resolución del conflicto intencional me enseñó cómo Git marca las diferencias y cómo se deben tomar decisiones conjuntas para unificar el código. Finalmente, configurar GitHub Actions representó un gran avance en mi comprensión de la integración continua y su papel en proyectos de desarrollo profesional.

Antony Guamán:

Mediante esta práctica pude experimentar directamente el flujo de trabajo que utilizan los equipos de desarrollo profesionales. Entendí la importancia de trabajar en ramas separadas para mantener el código organizado y estable en todo momento. La creación del archivo test.yml me permitió comprender cómo se configura un workflow de GitHub Actions y qué pasos ejecuta automáticamente al hacer un push. Ver el error 'All checks have failed' cuando ambos editamos la misma línea fue muy ilustrativo sobre lo que ocurre en proyectos reales sin coordinación. Resolver el conflicto y ver el repositorio final con ambos aportes integrados me dio confianza para trabajar colaborativamente en futuros proyectos de software.

Bibliografía

Chacon, S. & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

GitHub, Inc. (2024). GitHub Documentation. <https://docs.github.com>

Fowler, M. (2006). Continuous Integration. <https://martinfowler.com/articles/continuousIntegration.html>

Humble, J. & Farley, D. (2010). Continuous Delivery. Addison-Wesley.

Kim, G., Humble, J., Debois, P. & Willis, J. (2016). The DevOps Handbook. IT Revolution Press.

Atlassian. (2024). Git Branch. <https://www.atlassian.com/git/tutorials/using-branches>

Torvalds, L. (2005). Git — Fast Version Control System. <https://git-scm.com>